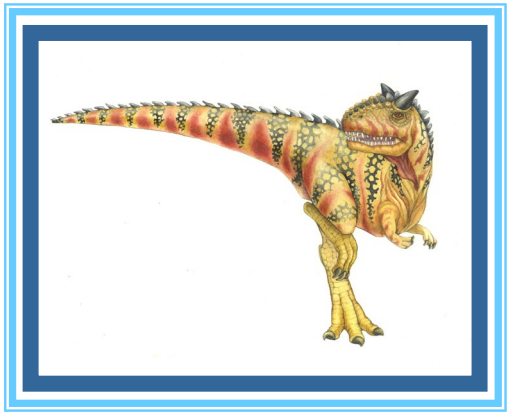
Chapter 9: Virtual Memory



Silberschatz, Galvin and Gagne ©2013



Chapter 9: Virtual Memory

- 9.1 Background
- 9.2 Demand Paging
- 9.4 Page Replacement
- 9.5 Allocation of Frames
- 9.6 Thrashing
- 9.11 Summary





Objectives

- To describe the benefits of a virtual memory system.
- To explain the concepts of demand paging, page-replacement algorithms, and allocation of page frames.
- To discuss the principle of the working-set model.
- To examine the relationship between shared memory and memory-mapped files
- To explore how kernel memory is managed.





9.1 Background

- Code needs to be in memory to execute, but entire program rarely used.
- Entire program code not needed at same time.
- Consider ability to execute partially-loaded program:
  - ✓ Program no longer constrained by limits of physical memory.
  - ✓ Each program takes less memory while running; hence more programs run at the same time.
    - Increased CPU utilization and throughput with no increase in response time or turnaround time.
  - ✓ Less I/O needed to load or swap programs into memory; hence, each user program runs faster.



Background

Concepts

Virtual memory – separation of user logical memory from physical memory:

- Only part of the program needs to be in memory for execution.
- Logical address space can therefore be much larger than physical address space
- Allows address spaces to be shared by several processes.
- Allows for more efficient process creation.
- More programs running concurrently.
- Less I/O needed to load or swap processes.



Background

Virtual address space of a process refers to the logical (or virtual) view how it stored in memory.

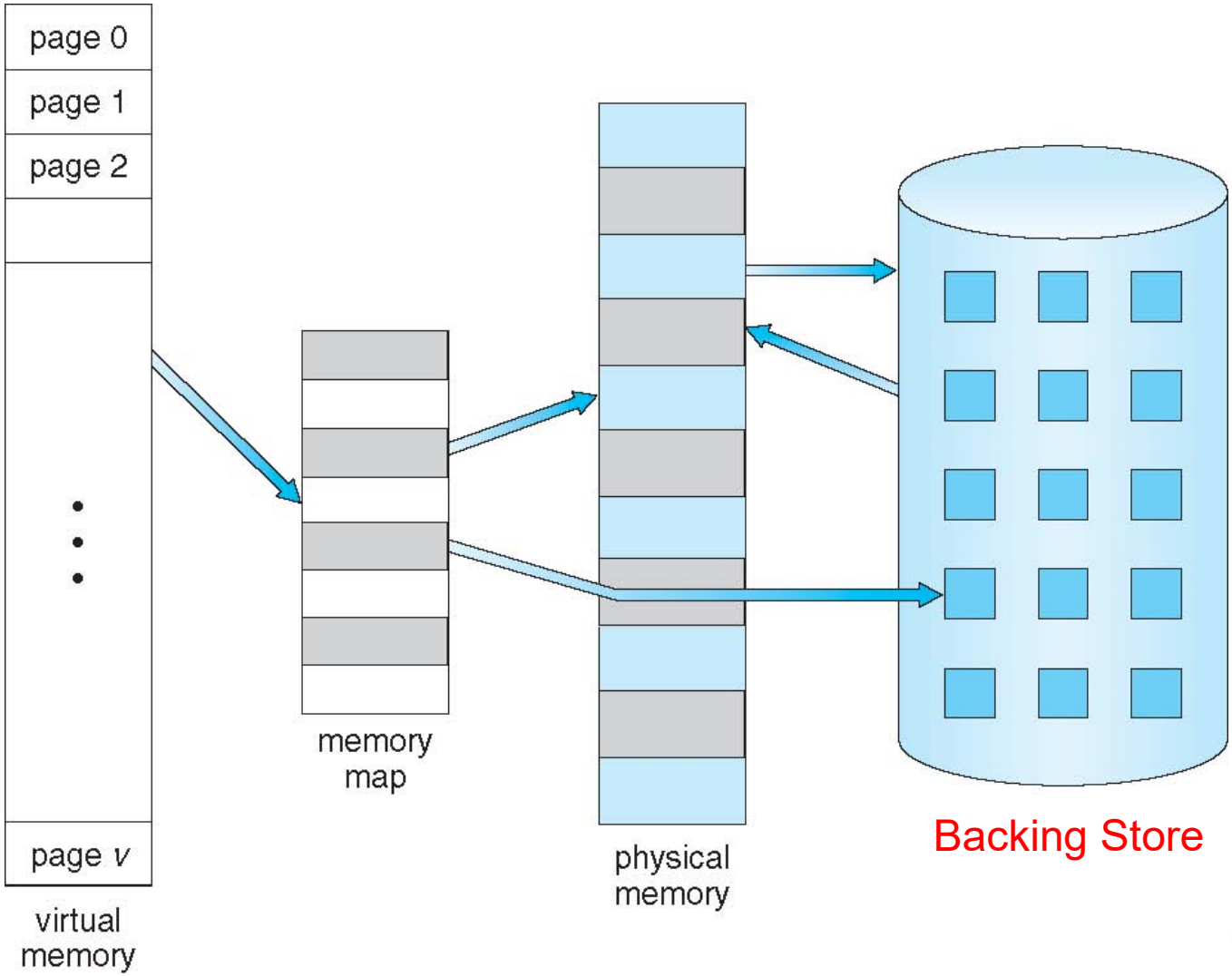
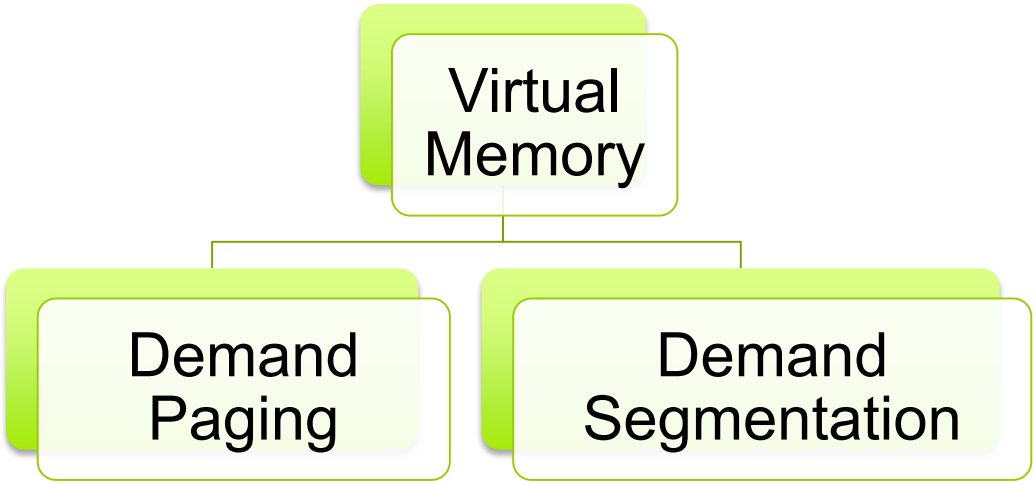


Figure 9.1 Virtual Memory That is Larger Than Physical Memory.



Background

- **Virtual memory** can be implemented via TWO approaches:



(Not cover in this course)



Virtual Memory: Demand Paging



Demand Paging

Overview

- Allowed for wide availability of **virtual memory** concept:
  - Provides appearance of almost infinite or nonfinite physical memory.
  - Jobs run with less main memory than required in paged memory allocation scheme.
- **Disadvantages:**
  - Increased overhead caused by tables and page interrupts.
  - Requires high-speed direct access storage device.

Solution: Swapping

- How and when pages passed in memory?
- Depends on predefined policies.



9.2 Demand Paging

A strategy to implement virtual memory

- Bring a page into memory only when it is needed:
  - Less I/O needed
  - Less memory needed
  - Faster response
  - More users

Lazy swapper

- never swaps a page into memory unless page will be needed
- Swapper that deals with pages is a **pager**

- **Page** is needed → **reference** to it
  - invalid reference → abort
  - not-in-memory → bring to memory

Demand Paging

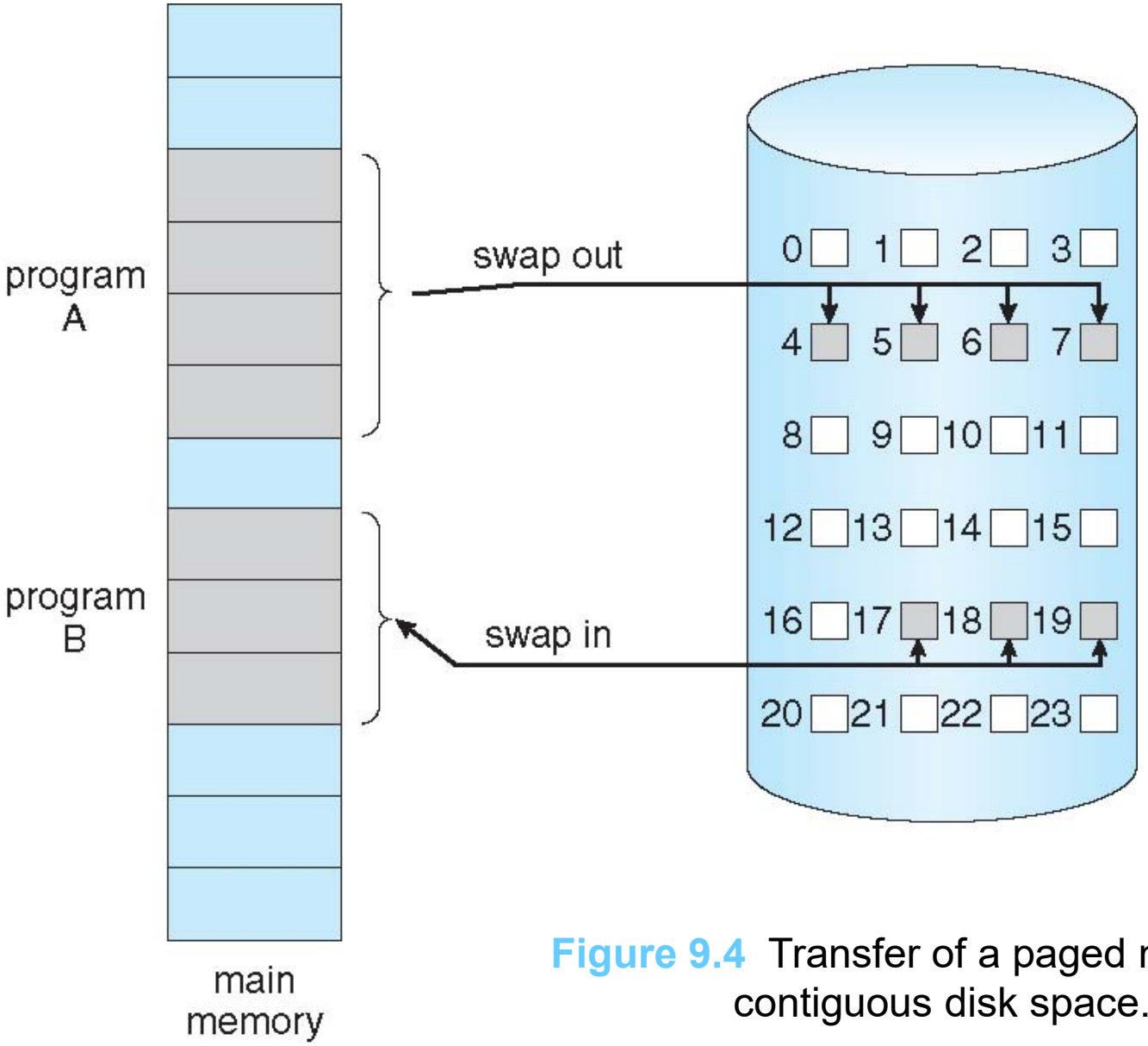


Figure 9.4 Transfer of a paged memory to contiguous disk space.



Demand Paging

Basic concepts

- When a process is to be swapped in, the pager guesses which pages will be used before the process is swapped out again.
- Instead of swapping in a whole process, the pager brings in only those "guessed" pages into memory.
- Need new MMU functionality to implement demand paging.

  - ✓ Need to distinguish between the pages that are in memory and the pages that are on the disk.
  - ✓ Use a variation of the valid-invalid scheme used for protection (see next slide).



Demand Paging

- If pages needed are already **memory resident.**
  - No difference from non demand-paging.

- If page needed and **not memory resident**, need to detect and load the page into memory from storage.
  - Without changing program behavior.
  - Without programmer needing to change code.



Demand Paging

Valid-Invalid Bit

- With each page table entry a **valid**–**invalid** bit is associated:

(v → in-memory

i → not-in-memory)

- Initially valid–invalid bit is set to i on all entries.



Demand Paging

Invalid → the page either:

- **not valid** (not in the logical address space of the process), or
- **valid** but is currently on the disk.

Access to a page marked invalid causes a page fault.

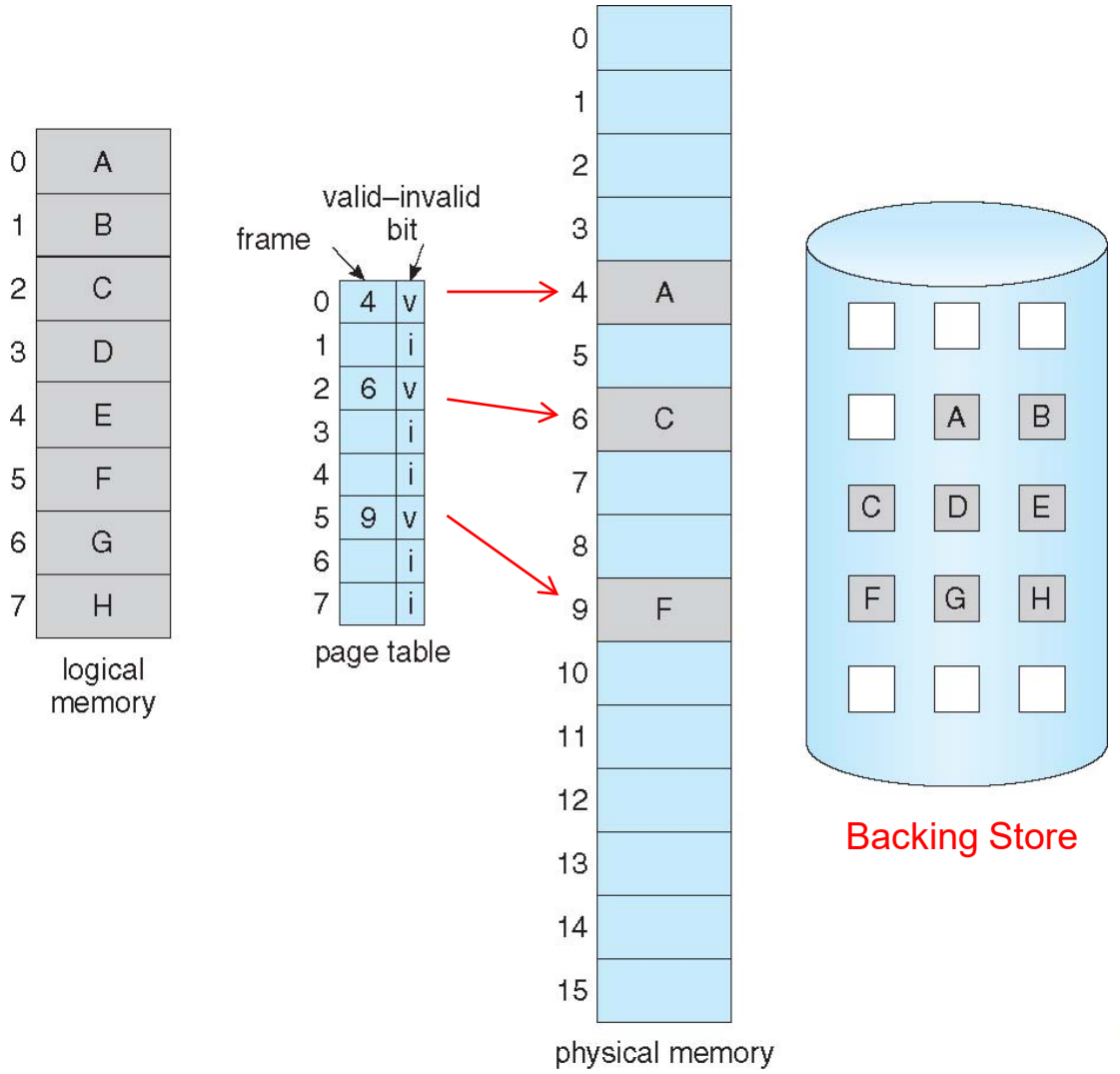


Figure 9.5 Page Table When Some Pages are Not in Main Memory



Demand Paging

Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system → page fault
- Actions when a page fault occurs:

① Operating system looks at another table to decide:
   - **Invalid** reference → abort

② **Valid** but not in memory → Go to step (3).

③ Find **free frame** in memory;

④ **Swap page** into frame via scheduled disk operation;

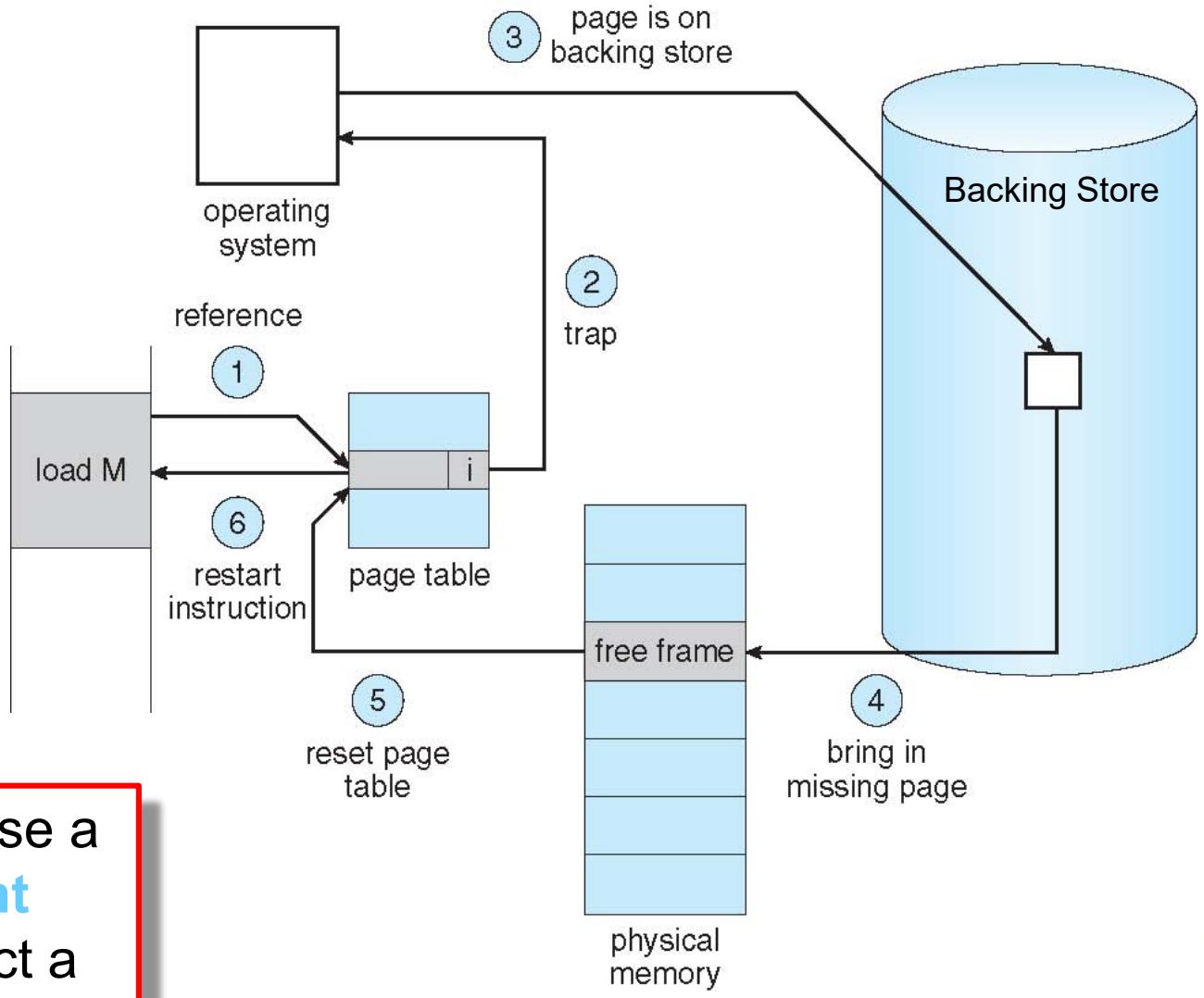
⑤ **Reset tables** to indicate page now in memory;
Set validation bit = v;

⑥ **Restart** the instruction.



Demand Paging

Trap → result of the OS's failure to bring the desired page into memory.



If no free frame, use a **page replacement algorithm** to select a **victim** frame.

Figure 9.6 Steps in Handling a Page Fault.



Page Replacement

Overview

- **Page replacement** occurs when:
  - A **page fault** occurs and we need to bring the desired page into memory.
  - There are NO free frames. to select a victim frame

- **Page replacement** – find page in memory, but not really in use, page it out:
  - Algorithm – decide which victim frame to free.
  - Performance – need an algorithm which will result in minimum number of page faults.

Same page may be brought into memory several times.

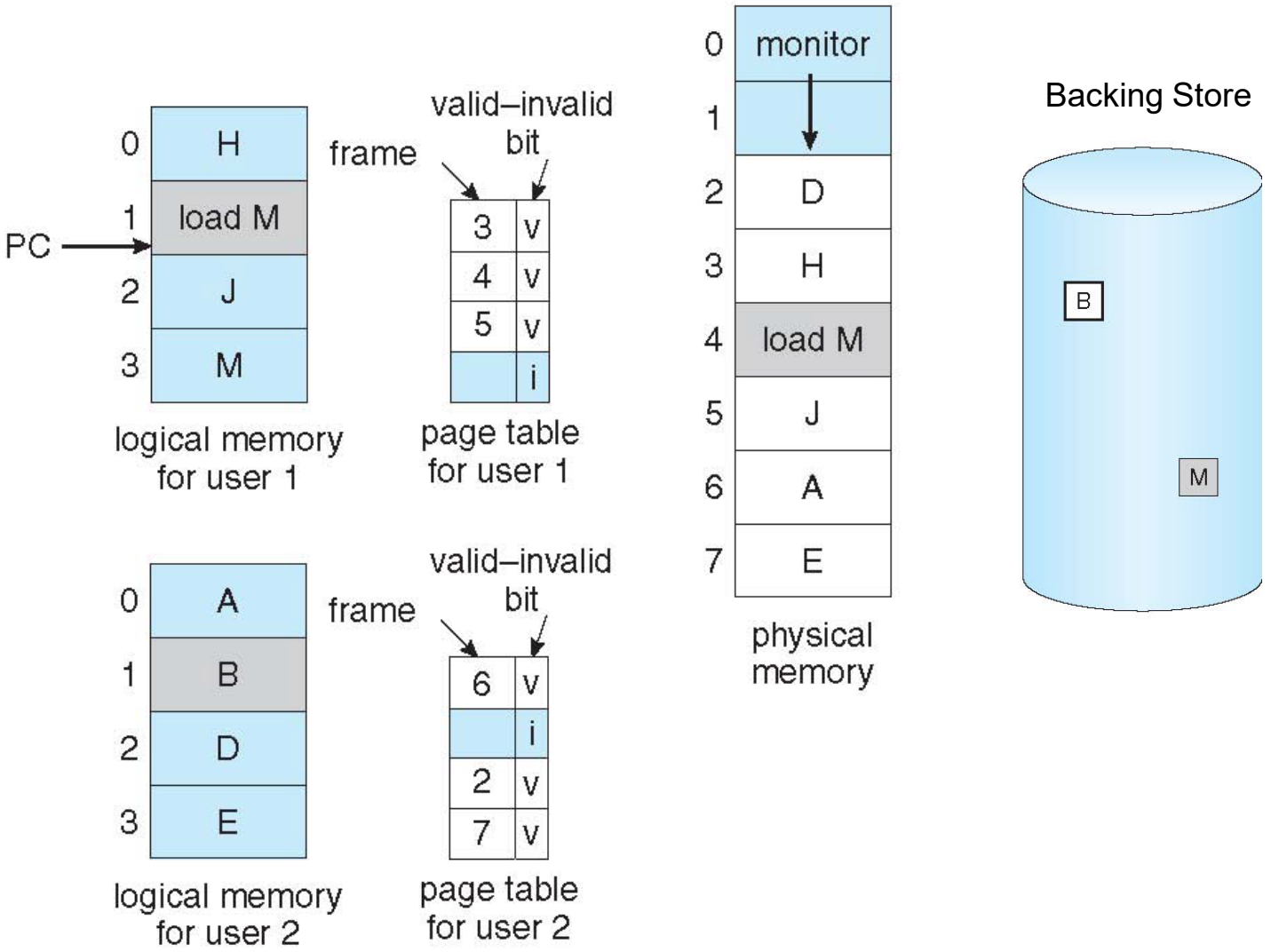


9.4 Page Replacement

Example

All frames are used, no free fames, user 2 needs B.

(Page: 402)





Page Replacement

Basic page replacement

① Find the location of the desired page on disk;

② Find a free frame:
- If there is a free frame, use it;
- If there is no free frame, use a page replacement algorithm to
  - Select a **victim** frame;
  - Write victim frame to disk if dirty

③ Bring the desired page into the (newly) free frame; update the page and frame tables

④ Restart the process that caused the page fault;



9.4 Page Replacement

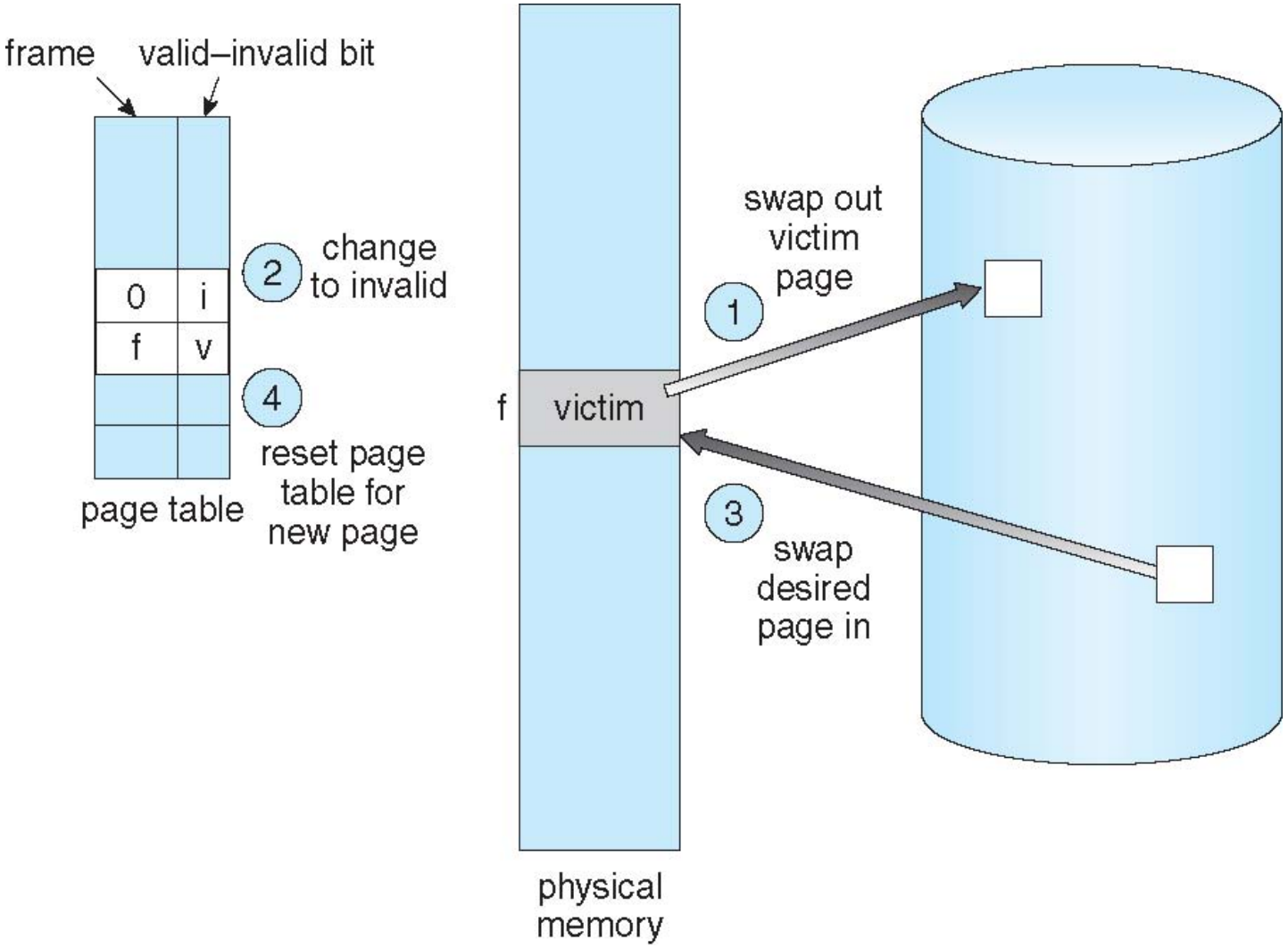


Figure 9.10 Page replacement.



Page Replacement

Problems: Overhead

If no free frames → TWO page transfers (one out one in):

- ✓ Doubles the page-fault service time.
- ✓ Increases access time.

Solution:

Use **modify bit** (or **dirty bit**) to reduce overhead of page transfers:

- ✓ only modified pages are written to back to disk.

- **Page replacement** completes separation between logical memory and physical memory
  - large virtual memory can be provided on a smaller physical memory.



Page Replacement

Page-Replacement Algorithms

- **Frame-allocation algorithm:** determines
  - ✓ How many frames to give each process? 几个要给.
  - ✓ Which frames to replace? 几个代替

- **Page-replacement algorithm:**
  - ✓ need an algorithm which will result in minimum number of **page faults**.

Algorithm
- minimum number of page faults.



Page Replacement

Page-Replacement Algorithms

- Evaluate algorithm by running it on a particular string of memory references (**reference string**) and compute the number of **page faults** on that string.

  - String is just page numbers, not full addresses.
  - Repeated access to the same page does not cause a **page fault**.
  - Results depend on **number of frames** available.



Page Replacement

If the number of frames increases, then the number of page faults decreases.

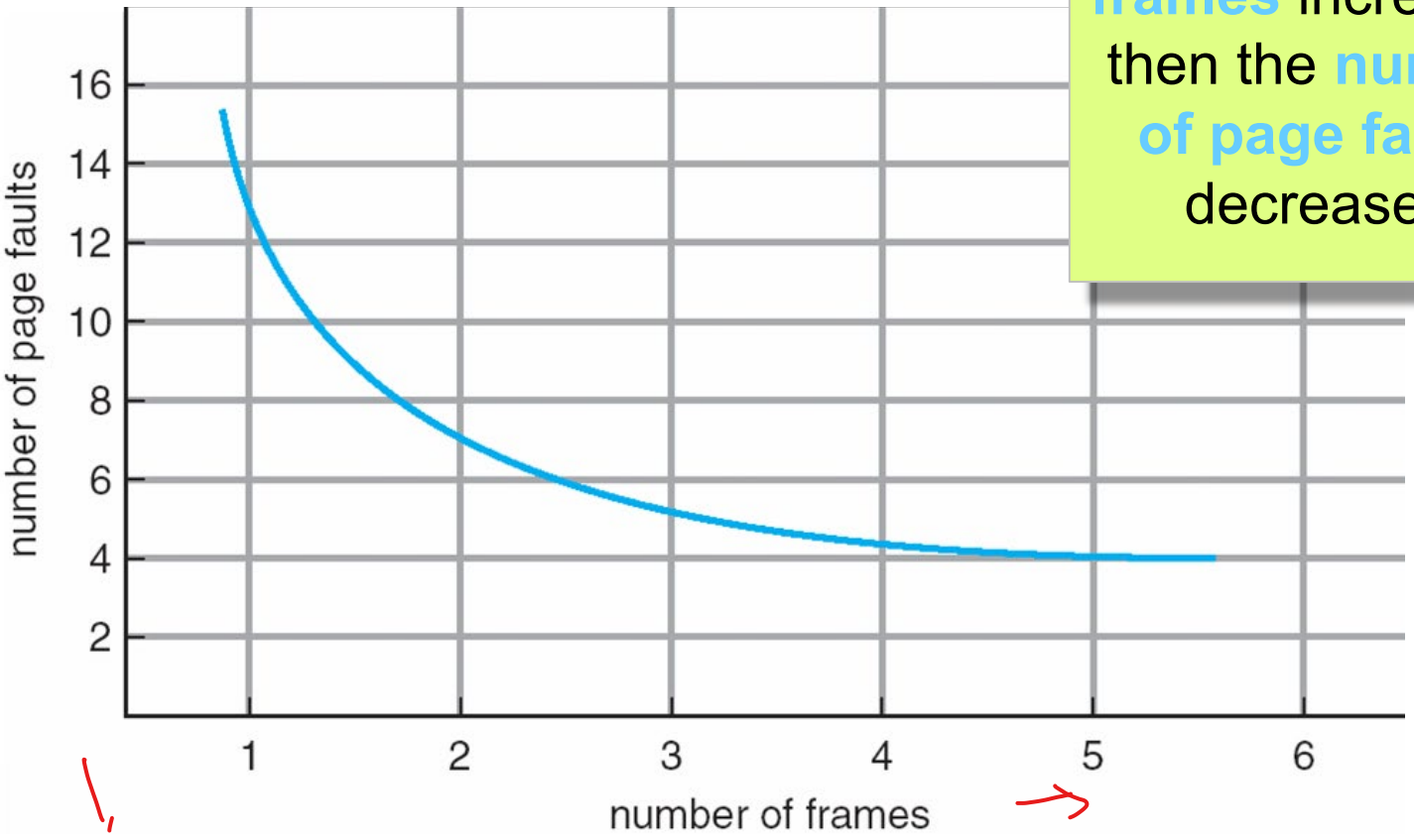


Figure 9.11 Graph of Page Faults Versus the Number of Frames.



Page Replacement

Reference String

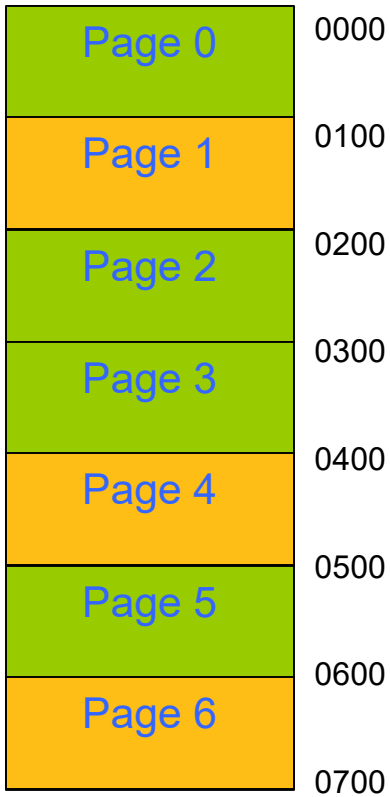
- If CPU accesses the following addresses:

0100 0432 0101 0612 0102
0103 0104 0101 0611 0105
0103 0104 0101 0610 0107
0103 0104 0101 0609 0101

0 4 1 6 1 1 1 1 6 1 1

If page size = 100 byte, reference string will be:

1 4 1 6 1 6 1 6 1 6 1





Page Replacement

Example

- Reference String:

  1 4 1 6 1 6 1 6 1 6 1

- Number of frames = 3
- What is the total number of page fault?

Solution

	1	4	1	6	1	6	1	6	1	6	1
Frame #0	1	1	1	1							
Frame #1		4		4							
Frame #2				6							

- Number of page fault = 3



9.4 Page Replacement

Exercise

Given a reference string:

1 4 1 6 1 6 1 6 1 6 1

If the number of frame is 1 and 2 respectively, define how many number of page faults will be generated for each of them. Show your works.

	1	4	1	6	1	6	1	6	1	6	1
Frame #0	1	4	1	6	1	6	1	6	1	6	1

– Number of page fault = 11

1 4 1 6 1 6 1 6 1 6 1

Frame #0: 1 1 H 6 6 H H H H
Frame #1: 4 4 1 H H H

– No. of page fault = 4





Page Replacement

- Policy to select page removal:
  - Crucial to system efficiency.

- Page replacement polices:
  - **First-In First-Out (FIFO)** policy

    → Best page to remove is one in memory longest.
  - **Optimal Page Replacement (OPT)** policy

    → Best page to remove is one not be accessed longest.
  - **Least Recently Used (LRU)** policy

    → Best page to remove is least recently accessed.



9.4 Page Replacement

- Another Page Replacement Policies:
  - **Random Page Replacement -** Replace page at random.
  - **LRU Approximation Algorithms**
    - Additional Reference Bits Algorithm
    - Second Chance Algorithm
    - Enhanced Second Chance Algorithm
  - **Counting-Based Replacement**
    - Least Frequently Used (LFU) - replaces page with smallest count
    - Most Frequently Used (MFU) -based on the argument that the page with the smallest count was probably just brought in and has yet to be used





Page Replacement

(a) First-In First-Out (FIFO)

- Removes the page that in memory with the longest of time;
- **Efficiency**
  - Ratio of page interrupts to page requests.
  - Not so good
- **FIFO anomaly**
  - More memory does not lead to better performance.



Page Replacement

Example

(Page: 406)

- FIFO.
- Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

- 3 frames = 3 (3 pages can be in memory at a time per process)
- What is the total number of page fault?

	7	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**
Frame #0	7	7	7	2		2	2	4	4	4	0			0	0			7	7	7
Frame #1		0	0	0		3	3	3	2	2	2			1	1			1	0	0
Frame #2			1	1		1	0	0	0	3	3			3	2			2	2	1

- Number of page fault = 15

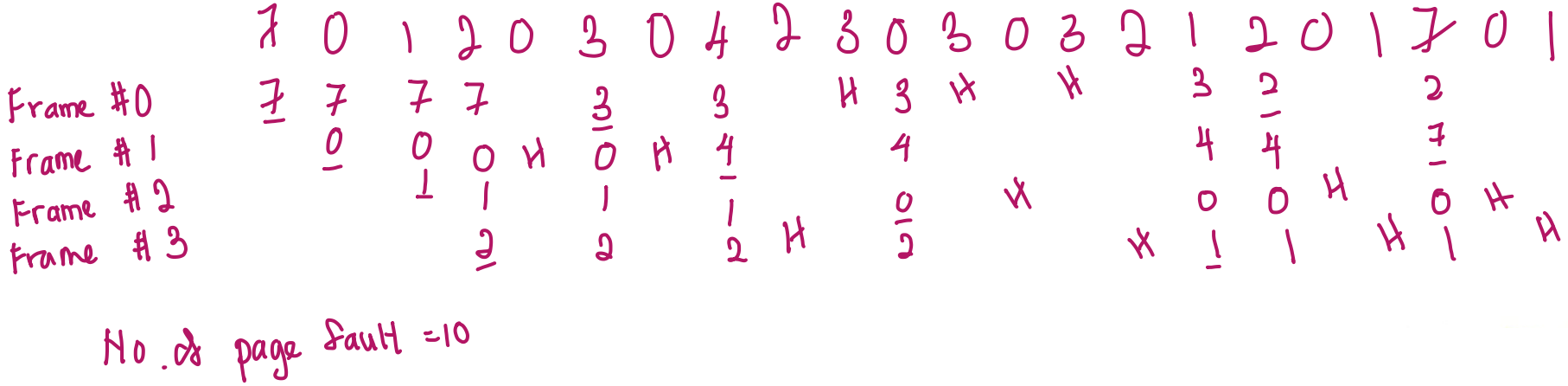
Page Replacement

Exercise

- FIFO.
- Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

- Number of frames = 4
- What is the total number of page fault?



No. of page fault = 10



Page Replacement

Number of Frames vs Page Faults

- One would expect that the more frames are allocated to a process the fewer page faults.

Exercise

Consider the reference string:

1 2 3 4 1 2 5 1 2 3 4

a) How many page faults if we have 3 frames?

b) How many page faults if we have 4 frames?



Reference String: 1 2 3 4 1 2 5 1 2 3 4

- 3 frames (3 pages can be in memory at a time per process)
- How many page faults?

Solution: Number of Page faults = 9

Frame #	1	2	3	4	1	2	5	1	2	3	4
1	1	1	1	4	4	4	5			5	5
2		2	2	2	1	1	1			3	3
3			3	3	3	2	2			2	4





Reference String: 1 2 3 4 1 2 5 1 2 3 4

- 4 frames (4 pages can be in memory at a time per process)
- How many page numbers?

Solution: Number of Page faults = 9

Frame #	1	2	3	4	1	2	5	1	2	3	4
1	1	1	1	1			5	5	5	5	4
2		2	2	2			2	1	1	1	1
3			3	3			3	3	2	2	2
4				4			4	4	4	3	3



Page Replacement

Belady's Anomaly

- If we use FIFO for page replacement, we can encounter the anomaly that more frames may lead to more page faults.
- FIFO Illustrating **Belady's Anomaly**

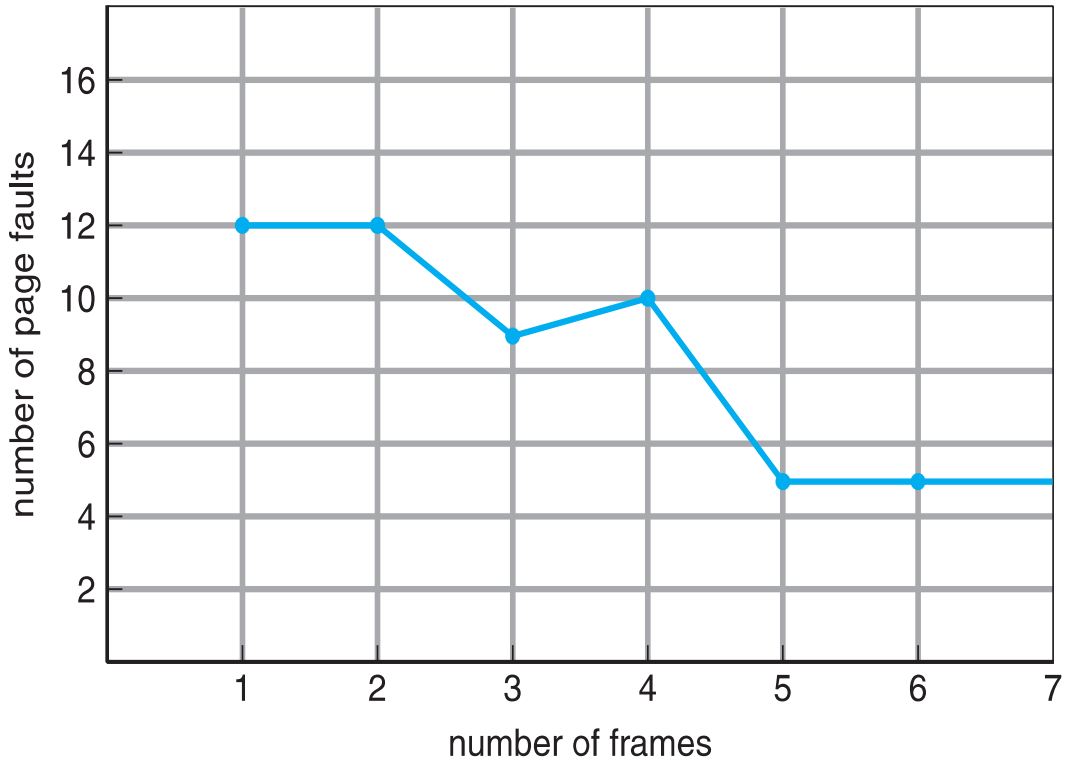


Figure 9.13 Page-fault curve for FIFO replacement on a reference string.



9.4 Page Replacement

(b) Optimal Page Replacement (OPT)

假设有用到的块 (有名词)

- Replace page that will not be accessed for longest period of time.

- **Q**: How do you know which page will not be accessed for longest period of time?
  - Can't read the future
- Used mainly for measuring how well a given algorithm performs.





Page Replacement

Example

(Page: 407)

- OPT.
- Reference String:

7 0 1 2 | 0 3 0 4 | 2 3 0 3 | 0 3 2 1 | 2 0 1 7 | 0 1

- 3 frames = 3 (3 pages can be in memory at a time per process)
- What is the total number of page fault?

	7	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**
Frame #0	7	7	7	2		2		2		2		2		2				7		
Frame #1		0	0	0		0		4		0		0		0				0		
Frame #2			1	1		3		3		3		1						1		

- Number of page fault = 9



Page Replacement

Exercise

- OPT.
- Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

- Number of frame = 4
- What is the total number of page fault?

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

Frame #0	7	7	7	7		3		3						1				1		
Frame #1		0	0	0		0		0						0				0		
Frame #2			1	1		1		4						4				7		
Frame #3				2		2		2						2				2		

No. of page fault = 8



Page Replacement

(c) Least Recently Used (LRU)

有前瞻

- Use past knowledge rather than future.
- Replace page that has not been used for the longest period of time.
- Associate time of last use with each page.





Page Replacement

Example

(Page: 407)

- LRU.
- Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

- 3 frames = 3 (3 pages can be in memory at a time per process)
- What is the total number of page fault?

	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame #0	7	7	7	2		2		4	4	4	0			1		1		1		
Frame #1		0	0	0		0		0	0	3	3			3		0		0		
Frame #2			1	1		3		3	2	2	2			2		2		7		

- Number of page fault = 12



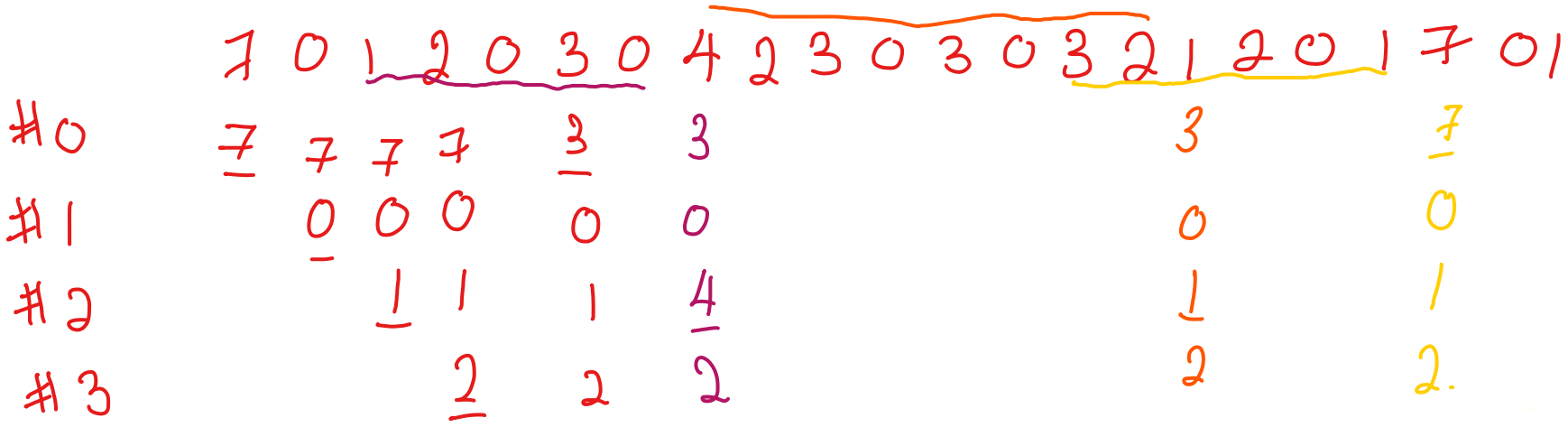
Page Replacement

Exercise

- LRU.
- Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

- Number of frame = 4
- What is the total number of page fault?



No. of page fault = 8

Page Replacement

LRU Implementation

- **Problem**: To determine an order for the frames defined by the time of last use.
- How to implement LRU algorithm?
- Two implementations:

① Counter

- Every page table entry has a counter associated with it;
- Every time page is referenced through this entry, the content of the clock if copied into the counter
- We replace the page with the smallest time value – search through table needed



Page Replacement

② Stack

- keep a stack of page numbers in a double link form: (*Header pointer* & *Tail pointer*)
- When a page is referenced:
  - move page to the **top**
  - Therefore, most recently used page is on **top** while least recently used is always at the **bottom**.
- No search for replacement

LRU needs special hardware and is still slow
LRU and OPT are cases of **stack algorithms** that don't suffer from Belady's Anomaly



Page Replacement

Example

(Page: 409)

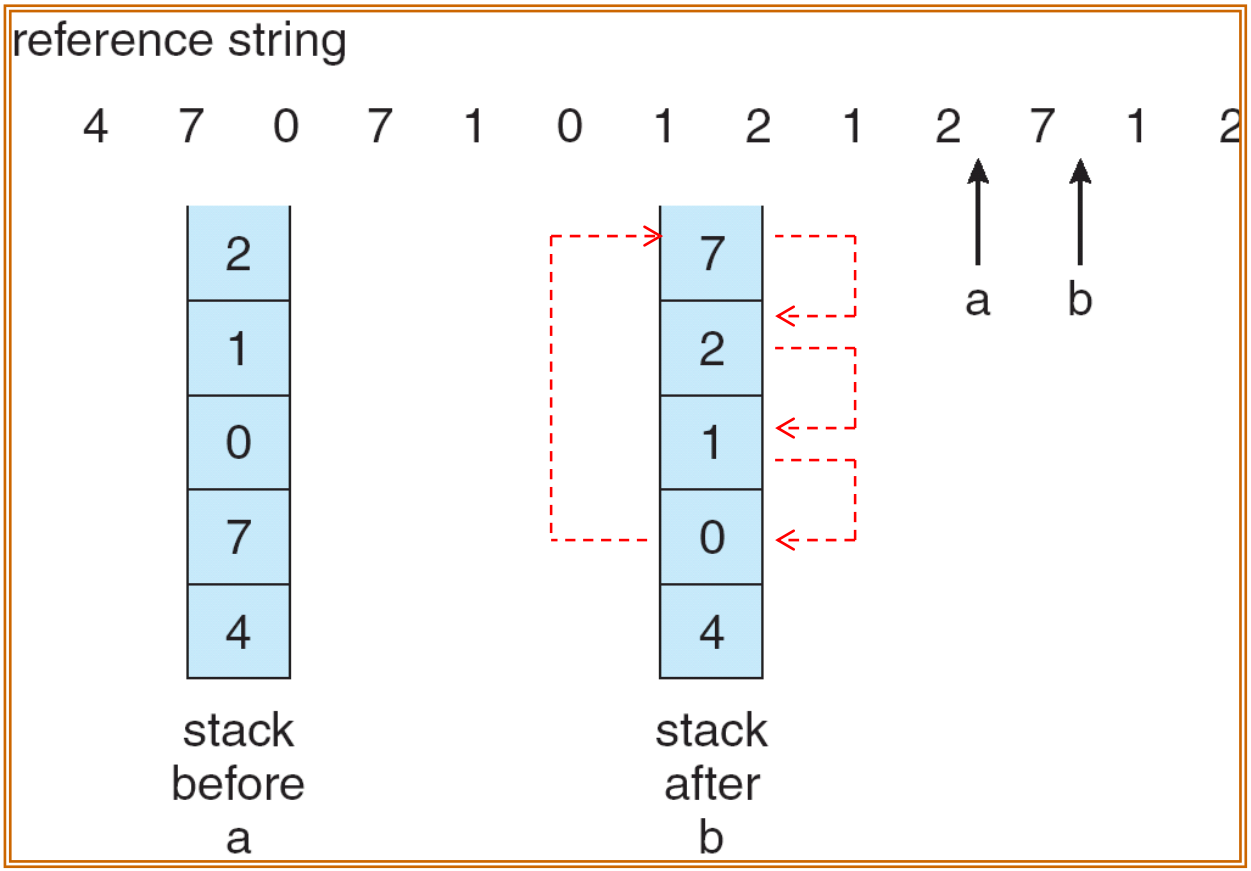


Figure 9.16 Use of a stack to record the most recent page references.



Virtual Memory: Demand Paging

(Some Issues)



Page Replacement

- **Global Replacement:**
  - Choose frames from all sets of frames in the system;
  - Process cannot control page-fault rate by itself;
- **Local Replacement:**
  - Choose frames already allocated to the process;
  - Effect – a process may keep pages that are seldom used 很少使用 while other process that need more frames has a high page fault rate;
- Generally, global replacement results in greater system throughput and is therefore the more common method.

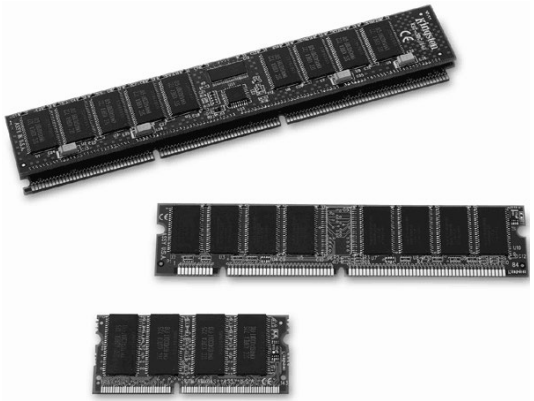




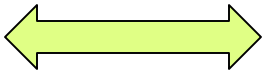
Thrashing

Overview

- An excessive amount of page swapping or paging activity (in & out) between **main memory** & **secondary storage**;
- A process is thrashing if it spends more time on paging than executing;



Main memory: DRAM





Secondary storage: Hard disk



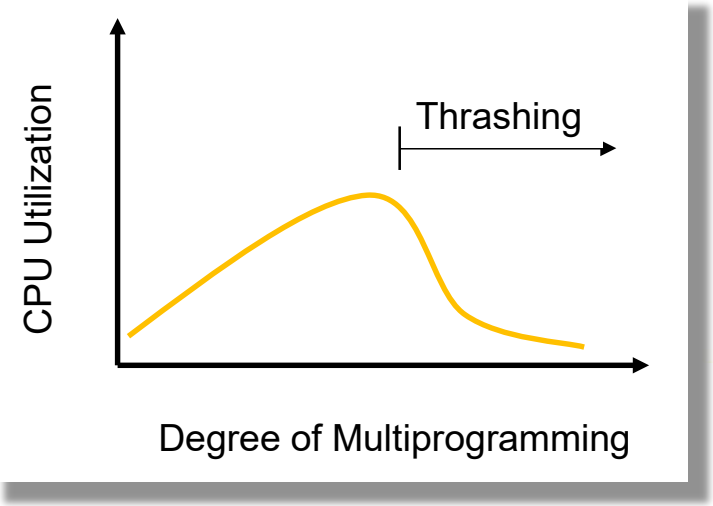
Thrashing

Why does this happen?

- If CPU utilization is low, OS increase the degree of multiprogramming;
- The number of programs increases while the number of frames remains the same;
- Therefore, page fault will occur more frequently;

Consequences:

- that leads to low CPU utilization;
- inefficient operation of system;





Thrashing

Example

Page fault due to program structure.

- Program structure
  - `Int[128,128] data;`
  - Each row is stored in one page
  - **Program 1**:

```
for (j = 0; j <128; j++)
     for (i = 0; i < 128; i++)
          data[i,j] = 0;
```

128 x 128 = 16,384 page faults

  - **Program 2**:

```
for (i = 0; i < 128; i++)
     for (j = 0; j < 128; j++)
          data[i,j] = 0;
```

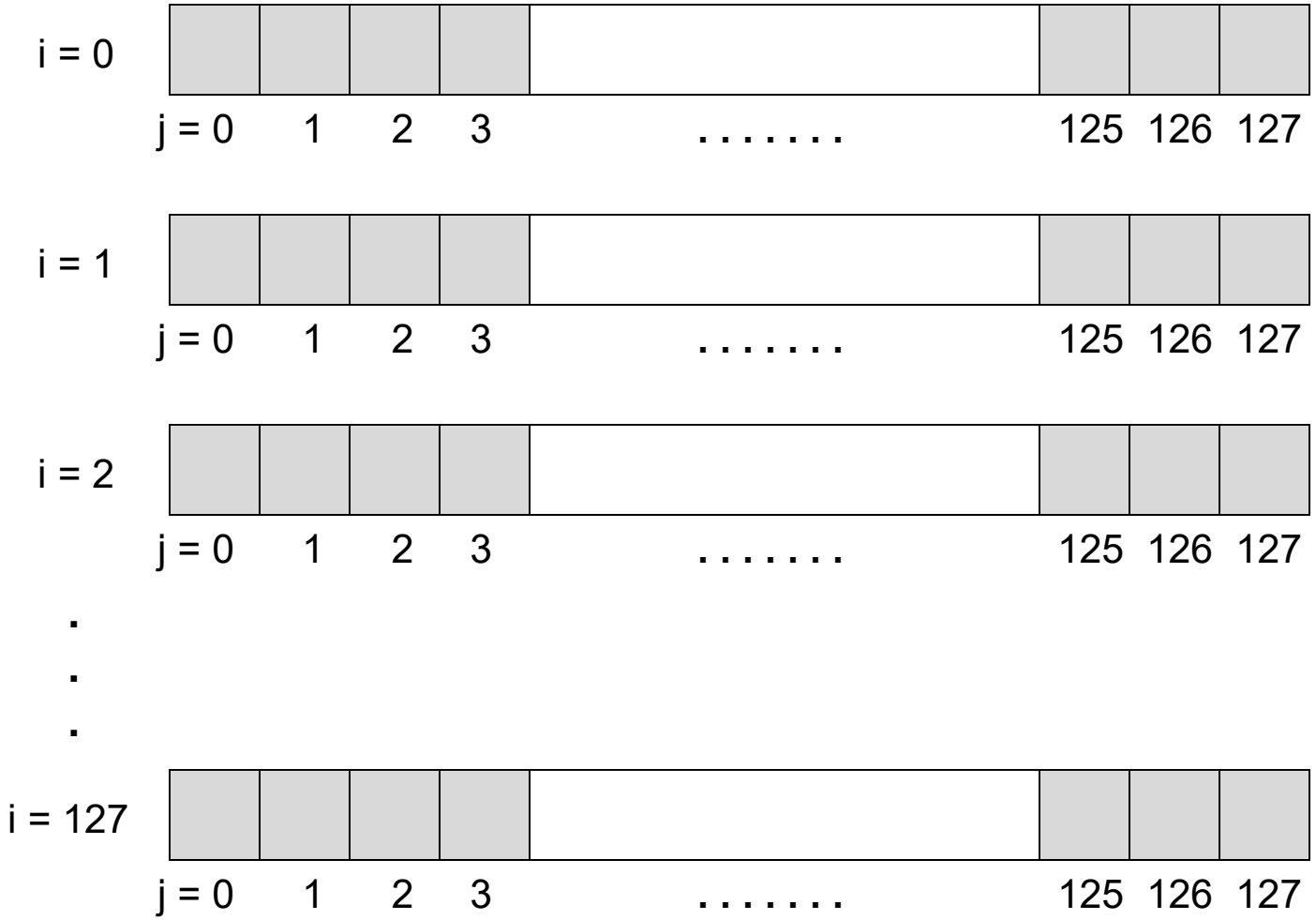
128 page faults



Thrashing

Program 1:

Page Fault(s) = **16384**

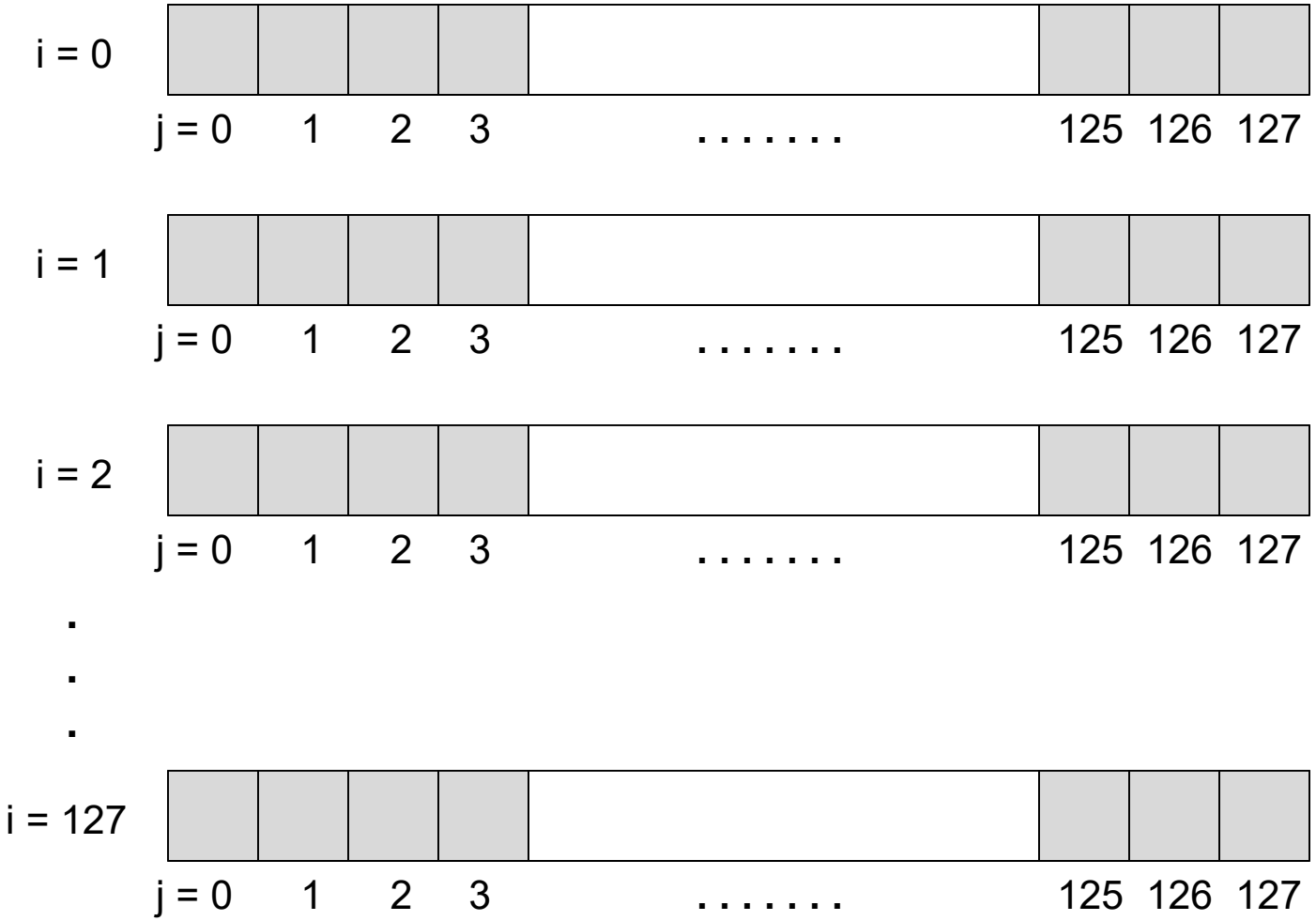




Thrashing

Program 2:

Page Fault(s) = **128**





Thrashing

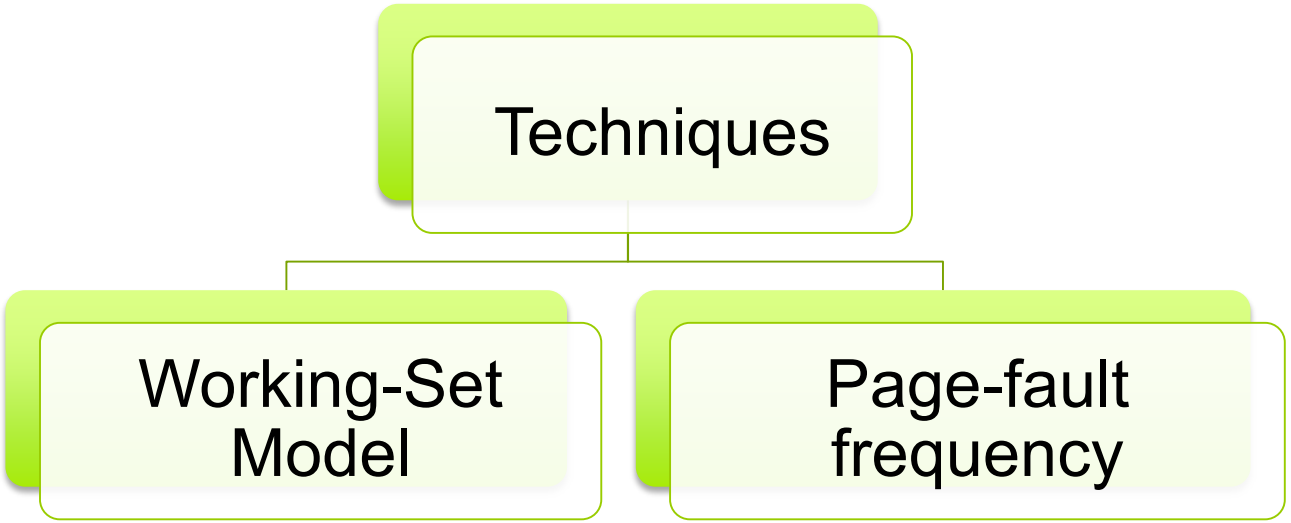


Figure Two techniques to prevent thrashing





Thrashing

(a) Working-Set (WS) Model

- Based on the assumption of locality;
- Uses a parameter, Δ , to define the **working-set window**;
- Δ is a fixed number of page references
- **Working-Set (WS)** → is defined to be the total number of pages referenced in the most recent Δ



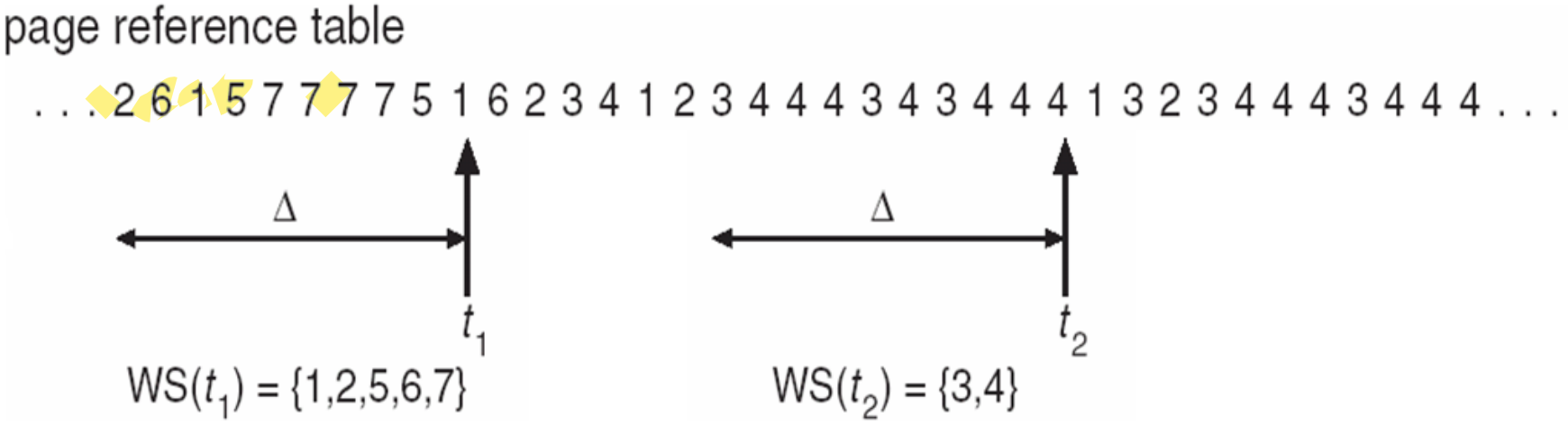


Thrashing

Example

(Page: 420)

- Given a sequence of memory references below;
- If Δ = 10memory references,



Solution

- The working set at t_1 is $\{1, 2, 5, 6, 7\}$
- By the time t_2, the working set has changed to $\{3, 4\}$



Thrashing

- System decides:
  - **Number of pages** comprising working set;
  - **Maximum number of pages** allowed for a working set;

- The accuracy of a working-set depends on the selection of Δ (its size)
  - If Δ too **small** → not encompass the entire locality;
  - If Δ too **large** → may overlap page several localities;
  - if $\Delta = \infty \Rightarrow$ will encompass entire program



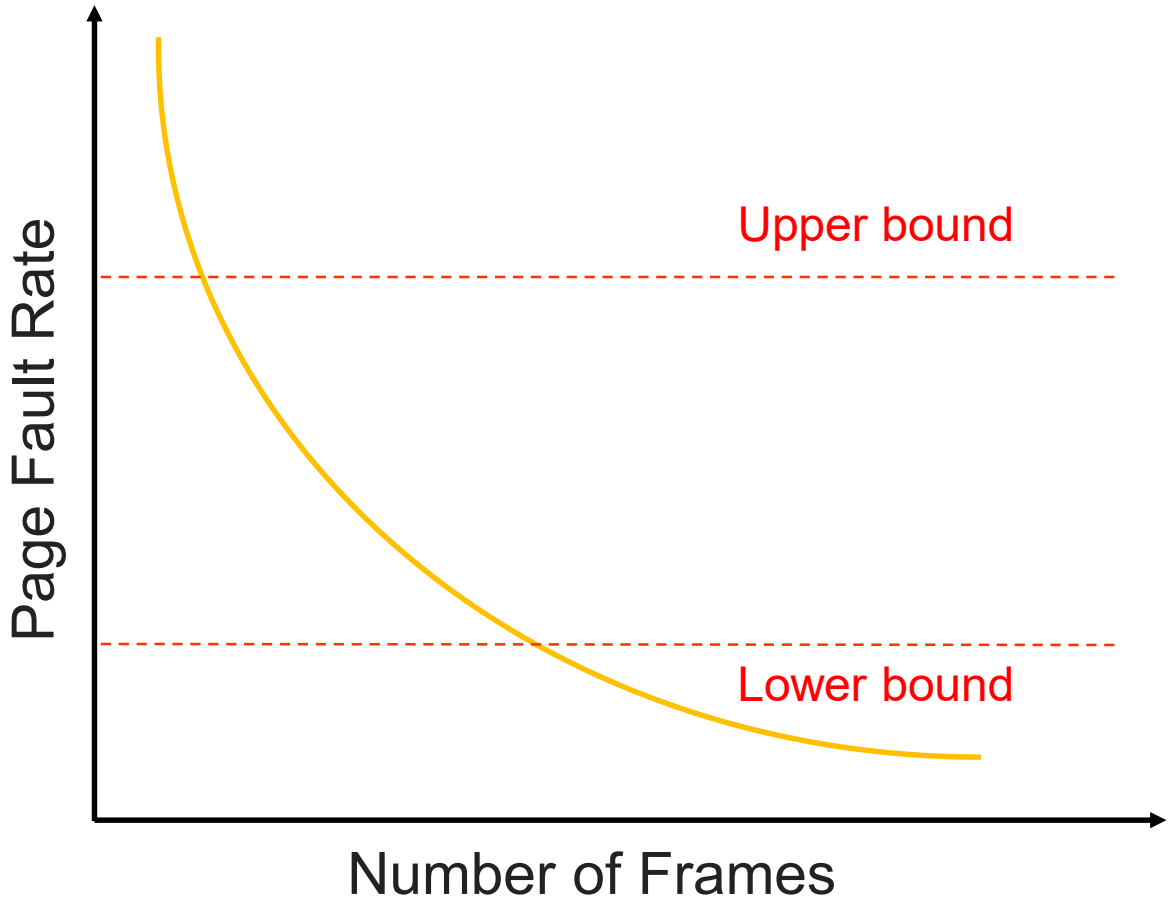
Thrashing

(b) Page-Fault Frequency (PFF)

- Establish "acceptable" **Page-Fault Frequency** rate and use local replacement policy:
  - If actual rate **too low**, process loses frame.
  - If actual rate **too high**, process gains frame.

- Provide **upper** and **lower** bound of page-fault.
- If **page-fault** rate > **upper** bound, allocate more frames.
- If **page-fault** rate < **lower** bound, reduce frame allocation.

Thrashing



Increase number of frames if the page fault rate is higher.

Decrease number of frames if the page fault rate is lower.

Figure 9.21 Page-fault frequency (PFF)



Summary

- It is able to execute a process with the **logical address space** larger than the available **physical address space**.
- **Virtual memory** allows :
  - ✓ to run extremely large processes.
  - ✓ to raise the degree of multiprogramming.
  - ✓ to increase CPU utilization.
- Virtual memory:
  - Programs execute if not stored entirely in memory.
  - Job's size no longer restricted to main memory size.



Summary

- **Demand paging** can be used to reduce the number of frames allocated to a process
- If total memory requirements exceed the capacity of physical memory, replace the pages from memory to free frames for new pages.
  - ✓ Various **page-replacement algorithms** are used.

  - ✓ **FIFO**: easy to program but suffers from Belady's anomaly.
  - ✓ **OPT**: requires future knowledge.
  - ✓ **LRU**: approximates the OPT, but difficult to implement.



Exercise

Consider the following page reference string:

1 2 3 4 2 1 5 6 2 1 2 3 7 6 3 2 1 2 3 6

How many **page faults** would occur for the following replacement algorithms, assuming **1, 3** and **5 frames**? Remember that all frames are initially empty.

a) FIFO replacement.

b) LRU replacement.

c) OPT replacement.





Answer(s): FIFO

1 frame: 20 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6

3 frames: 16 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	4		4	4	6	6	6		3	3	3		2	2		2	6
	2	2	2		1	1	1	2	2		2	7	7		7	1		1	1
		3	3		3	5	5	5	1		1	1	6		6	6		3	3

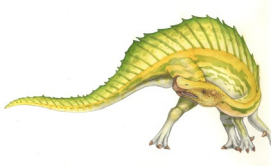




Answer(s): FIFO

5 frames: 10 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	6		6	6	6	6							
	2	2	2			2	2		1	1	1	1							
		3	3			3	3		3	2	2	2							
			4			4	4		4	4	3	3							
						5	5		5	5	5	7							





Answer(s): LRU

1 frame: 20 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6

3 frames: 15 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	4		4	5	5	5	1		1	7	7		2	2			2
	2	2	2		2	2	6	6	6		3	3	3		3	3			3
		3	3		1	1	1	2	2		2	2	6		6	1			6

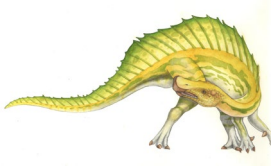




Answer(s): LRU

5 frames: 8 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	1				1	1		1					
	2	2	2			2	2				2	2		2					
		3	3			3	6				6	6		6					
			4			4	4				3	3		7					
						5	5				5	5		3					





Answer(s): OPT

1 frame: 20 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6

3 frames: 11 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	1				3	3			3	3			3
	2	2	2			2	2				2	7			2	2			2
		3	4			5	6				6	6			6	1			6





Answer(s): OPT

5 frames: 7 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	1					1							
	2	2	2			2	2					2							
		3	3			3	3					3							
			4			4	6					6							
						5	5					7							





Exercises

From Silberchatz, Operating System Concepts
Chapter 9 Exercises (page: 441 - 446)

- 9.5
- 9.6
- 9.8
- 9.9
- 9.11
- 9.12
- 9.17
- 9.19
- 9.20
- 9.21

